SBOM Summary

Software Bill of Materials inventory, package visibility, dependency evidence, vulnerability correlation, and release use cases.

Field	Value
Project	Enterprise DevSecOps Security Lab
Document Owner	Jagriti Banerjee
Document Type	SBOM Summary
Environment	Private Ubuntu VM, Docker, Kind Kubernetes, GitHub Actions, GHCR
Classification	Portfolio / Private Lab Evidence
Status	Final Detailed Spacing Fixed

Document Scope

This report is part of the Enterprise DevSecOps evidence package. It is written for technical reviewers and recruiters who want to understand how artifact trust, SBOM visibility, provenance, VEX notes, and exception governance were implemented and validated in the private lab.

Reviewer Focus Area	What this document proves
Inventory visibility	Explains SPDX and CycloneDX SBOM outputs and inventory counts.
Security triage	Maps SBOM inventory to scanner findings, VEX notes, and exceptions.
Release operations	Defines SBOM generation, quality checks, and retest expectations.

Evidence Package	Included
Supply chain controls	Cosign signing, Syft SBOM, provenance attestation, Gatekeeper metadata controls
Decision records	Security gate policy, exception rules, VEX status notes, retest expectations
Release quality focus	Artifact integrity, component transparency, vulnerability triage, and audit readiness

Executive Summary

The SBOM Summary documents the inventory created for the Enterprise DevSecOps image using Syft. The SBOM is a release evidence artifact that helps reviewers answer what components exist in the image, which components are vulnerable, which packages require updates, and which findings require VEX notes or exceptions.

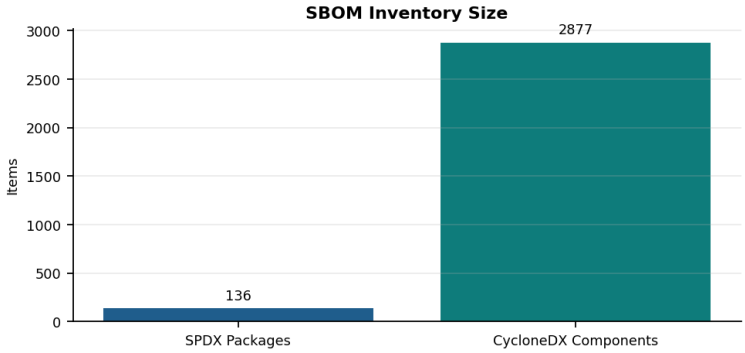

Figure 1 - SBOM inventory count comparison between SPDX and CycloneDX outputs.

Metric	Value	Interpretation
SPDX packages	136	Package-level inventory for license and vulnerability correlation.
CycloneDX components	2877	Component-rich representation including nested and transitive components.
SBOM formats	SPDX JSON, CycloneDX JSON	Supports multiple toolchains and compliance review preferences.
Generation tool	Syft	Creates SBOM from the container image and can output multiple formats.
SBOM consumers	Trivy, release reviewers, exception register, incident response	SBOM becomes the source inventory for triage and response.

SBOM Formats and Consumer Use

Format	Primary Strength	Consumer Use
SPDX JSON	Strong package/license data model and broad compliance familiarity.	Release evidence, license review, Cosign SBOM attachment.
CycloneDX JSON	Application security-oriented component modeling.	Dependency analysis, vulnerability correlation, inventory review.
Human-readable table	Quick reviewer understanding.	Portfolio screenshots and summary reports.

Generation and Verification Workflow

The SBOM is generated from the image rather than only from source files. This is important because the image includes runtime packages, transitive dependencies, and installed components that may not be obvious from the repository alone.

```
syft "$IMAGE" -o spdx-json=security/sbom/sbom-spdx.json
syft "$IMAGE" -o cyclonedx-json=security/sbom/sbom-cyclonedx.json
syft "$IMAGE" -o table | tee security/reports/sbom-table-report.txt
jq '.packages | length' security/sbom/sbom-spdx.json
cosign attach sbom --sbom security/sbom/sbom-spdx.json "$IMAGE"
cosign download sbom "$IMAGE" > security/reports/downloaded-sbom-spdx.json
```

SBOM Quality Controls

Quality Control	Expected Result	Risk If Missing
Non-empty inventory	Package/component count is greater than zero.	Empty SBOM creates false confidence.
Digest traceability	SBOM is tied to image digest.	SBOM may describe a different image.
Multiple formats	SPDX and CycloneDX are both available.	Tool interoperability is reduced.
Attestation	SBOM is signed or attached through Cosign.	SBOM could be replaced without detection.
Scanner correlation	Trivy/pip-audit findings map to package names.	Triage becomes slow and inconsistent.

SBOM-Driven Vulnerability Triage

Triage Step	Question	Output
Identify package	Is the vulnerable package present in the SBOM?	Package name and version.
Confirm scanner match	Does Trivy or pip-audit report the same package?	Advisory ID and severity.
Assess reachability	Is the package used by the app or runtime path?	VEX status candidate.
Decide action	Update, remove, mitigate, or document exception?	Remediation ticket or exception record.
Retest	Does a new scan confirm closure?	Updated SBOM and clean scan evidence.

Operational Retest Checklist

- Regenerate SBOM after any Dockerfile, base image, or requirements.txt change.
- Confirm image digest before and after SBOM generation.
- Update VEX notes when the SBOM inventory or vulnerability context changes.
- Attach SBOM to the image and verify it can be downloaded from the registry.
- Retain SBOM and scanner reports as release evidence artifacts.

References

- CISA VEX minimum requirements: https://www.cisa.gov/sites/default/files/2023-04/minimum-requirements-for-vex-508c.pdf
- OpenVEX specification: https://github.com/openvex/spec/blob/main/OPENVEX-SPEC.md
- SLSA specification: https://slsa.dev/spec/v1.2/
- SLSA provenance specification: https://slsa.dev/spec/v1.0/provenance
- Sigstore Cosign project: https://github.com/sigstore/cosign
- Cosign specifications and attestation support: https://docs.sigstore.dev/cosign/system_config/specifications/
- Sigstore project overview: https://www.sigstore.dev/